

With the OpenAI API, you can use a large language model to generate text from a prompt, as you might using ChatGPT. Models can generate almost any kind of text response—like code, mathematical equations, structured JSON data, or human-like prose.

Here's a simple example using the Responses API.

Generate text from a simple prompt

JavaScript 

```
1 import OpenAI from "openai";
2 const client = new OpenAI();
3
4 const response = await client.responses.create({
5   model: "gpt-6-astra",
6   input: "Write a one-sentence bedtime story about a unicorn.",
7 });
8
9 console.log(response.output_text);
```

An array of content generated by the model is in the `output` property of the response. In this simple example, we have just one output which looks like this:

```
1 [
2   {
3     "id": "msg_67b73f697ba4819183a15cc17d011509",
4     "type": "message",
5     "role": "assistant",
6     "content": [
7       {
8         "type": "output_text",
9         "text": "Under the soft glow of the moon, Luna the
10 unicorn danced through fields of twinkling stardust,
11 leaving trails of dreams for every child asleep.",
12         "annotations": []
13       }
14     ]
15   }
16 ]
```

 The `output` array often has more than one item in it! It can contain tool calls, data about reasoning tokens generated by reasoning models, and other items. It is not safe

Ask AI

Loading docs agent...

When in doubt, `gpt-6-astra` offers a strong default for general-purpose text generation and prompt iteration.

Prompt engineering

Prompt engineering is the process of writing effective instructions for a model, such that it consistently generates content that meets your requirements.

Because the content generated from a model is non-deterministic, prompting to get your desired output is a mix of art and science. However, you can apply techniques and best practices to get good results consistently.

Some prompt engineering techniques work with every model, like using message roles. But different model types (like reasoning versus GPT models) might need to be prompted differently to produce the best results. Even different snapshots of models

Loading docs agent...

```
5   model: "gpt-o-astra",  
6   reasoning: { effort: "low" },  
7   instructions: "Talk like a pirate.",  
8   input: "Are semicolons optional in JavaScript?",  
9 });  
10  
11 console.log(response.output_text);
```

The example above is roughly equivalent to using the following input messages in the `input` array:

Generate text with messages using different roles

JavaScript  

```
1  import OpenAI from "openai";  
2  const client = new OpenAI();  
3  
4  const response = await client.responses.create({
```

Loading docs agent...

application developer, prioritized
ahead of `user` messages.

user, prioritized behind
`developer` messages.

`assistant` role.

A multi-turn conversation may consist of several messages of these types, along with other content types provided by both you and the model. Learn more about [managing conversation state here](#).

You could think about `developer` and `user` messages like a function and its arguments in a programming language.

- `developer` messages provide the system's rules and business logic, like a function definition.
- `user` messages provide inputs and configuration to which the `developer` message instructions are applied, like arguments to a function.

Loading docs agent...

When writing `developer` and `user` messages, you can help the model understand logical boundaries of your prompt and context data using a combination of Markdown formatting and XML tags.

Markdown headers and lists can be helpful to mark distinct sections of a prompt, and to communicate hierarchy to the model. They can also potentially make your prompts more readable during development. XML tags can help delineate where one piece of content (like a supporting document used for reference) begins and ends. XML attributes can also be used to define metadata about content in the prompt that can be referenced by your instructions.

In general, a developer message will contain the following sections, usually in this order (though the exact optimal content and order may vary by which model you are using):

Loading docs agent...

```
6
7 # Instructions
8
9 * When defining variables, use snake case names (e.g.
10 my_variable)
11 instead of camel case names (e.g. myVariable).
12 * To support old browsers, declare variables using the older
13 "var" keyword.
14 * Do not give responses with Markdown formatting, just return
15 the code as requested.
16
17 # Examples
18
19 <user_query>
20 How do I declare a string variable for a first name?
21 </user_query>
22
23 <assistant_response>
```

Loading docs agent...

Instructions

- * Only output a single word in your response with no additional formatting or commentary.
- * Your response should only be one of the words "Positive", "Negative", or "Neutral" depending on the sentiment of the product review you are given.

Examples

```
<product_review id="example-1">
```

I absolutely love this headphones – sound quality is amazing!

```
</product_review>
```

```
<assistant_response id="example-1">
```

Positive

Loading docs agent...

The technique of adding additional relevant context to the model generation request is sometimes called **retrieval-augmented generation (RAG)**. You can add additional context to the prompt in many different ways, from querying a vector database and including the text you get back into a prompt, or by using OpenAI's built-in [file search tool](#) to generate content based on uploaded documents.

Planning for the context window

Models can only handle so much data within the context they consider during a generation request. This memory limit is called a **context window**, which is defined in terms of [tokens](#) (chunks of data you pass in, from text to images).

Models have different context window sizes from the low 100k range up to one million tokens for newer GPT-4.1 models. [Refer to the model docs](#) for specific context window sizes per model.

Loading docs agent...

benefit from very precise instructions.

You could think about the difference between reasoning and GPT models like this.

- A reasoning model is like a senior co-worker. You can give them a goal to achieve and trust them to work out the details.
- A GPT model is like a junior coworker. They'll perform best with explicit instructions to create a specific output.

For more information on best practices when using reasoning models, [refer to this guide](#).

Next steps

Loading docs agent...